

Polytechnique Montréal

Département de génie informatique et génie logiciel

Cours INF1900:
Projet initial de système embarqué

Travaux pratiques 7 et 8

Makefile et production de librairie statique

Par l'équipe

No 0110

Noms:

Nermine Morri
Felicia Remadji Libar
Kamal Chalhoub
Diaby Alpha Boubacar

Date:
27 octobre 2026

Partie 1 : Description de la librairie

La librairie statique `librobot.a` est construite dans le répertoire `tp/tp7/lib`. Elle regroupe le code des six premières semaines qui contrôle le matériel du robot et que chacun réécrivait d'un travail pratique à l'autre : DEL, bouton-poussoir, interruptions externes, minuteries, moteurs, communication RS232, conversion analogique/numérique et mémoire externe. Elle contient aussi le module de débogage du travail pratique 8. L'objectif est de configurer chaque périphérique en une ou deux lignes dans le code d'application, sans recopier de fichiers.

Lors de la mise en commun, nous avons conservé uniquement le code qui a un fort potentiel de réutilisation. Les machines à états, les routines d'interruption (ISR) et les variables globales restent dans le code d'application, car elles sont propres à un problème donné. Les fichiers .h ne contiennent que des déclarations de classes, de fonctions et de constantes.

Organisation et conventions

- Chaque concept est regroupé dans une classe simple, déclarée dans un fichier .h et définie dans un fichier .cpp. Le nom du fichier est celui de la classe, en minuscules (par exemple `Del` dans `del.h` et `del.cpp`), comme les fichiers `can` et `memoire_24` fournis par le cours.
- Chaque fichier .h est protégé par `#pragma once` et n'inclut que ce dont il a besoin. Les autres inclusions, comme `<util/delay.h>`, sont faites dans le fichier .cpp.
- Toutes les valeurs de configuration sont des constantes nommées (`static constexpr`) déclarées dans la classe : aucun nombre magique n'apparaît dans le code.
- Les registres de contrôle (TCCRn, EICRA, UCSR0B, etc.) ne sont jamais écrits avec `=`. On efface d'abord les bits concernés avec `&=`, puis on place les nouveaux avec `|=`, pour ne pas écraser la configuration faite par une autre classe.
- Les noms de classes, de méthodes et de constantes sont choisis pour que le code se lise sans commentaires. Les fichiers .cpp n'en contiennent donc aucun.

Répartition des minuteries

Une minuterie ne peut remplir qu'un seul rôle à la fois, et le PWM matériel ne sort que sur des broches fixes. Sur notre robot, chaque roue est reliée au pont en H par un connecteur de deux fils (E pour la vitesse, D pour la direction) qui occupe deux broches voisines de la carte. Les seules paires où la broche E peut recevoir un PWM de la même minuterie pour les deux roues sont PB2/PB3 et PB4/PB5, avec les sorties OC0A et OC0B de la minuterie 0. Nous avons donc réparti les minuteries ainsi :

Minuterie	Classe	Rôle et broches
0 (8 bits)	Moteurs	PWM des deux roues : E sur PB3 et PB4, D sur PB2 et PB5
1 (16 bits)	Minuterie1	Délais jusqu'à environ 8,3 s ; aucune broche
2 (8 bits)	Minuterie2	Fréquence du piézo sur PD7, PD6 servant de masse

La minuterie 1, la seule de 16 bits, reste ainsi libre pour les délais, et le robot peut rouler, mesurer le temps et produire un son en même temps. PB5 sert aussi à la programmation de la puce. Sur notre robot, la programmation fonctionne avec la roue droite branchée, mais nous la débranchons si `make install` échoue.

Classes de la librairie

Del : contrôle une DEL bicolore branchée sur deux broches d'un même port. Le constructeur reçoit un pointeur vers le port, un pointeur vers son registre de direction et les deux broches ; il les configure en sortie. `changerCouleur()` reçoit une couleur de l'énumération `Couleur` (ROUGE, VERTE, ETEINTE) et ne modifie que les deux broches de la DEL. `afficherAmbre()` alterne rouge et vert toutes les 5 ms ; appelée en boucle, elle donne l'ambre par persistance rétinienne.

Bouton : lit le bouton-poussoir par scrutation. Le constructeur met la broche en entrée, et `estAppuye()` relit la broche après 10 ms pour éliminer les rebonds mécaniques.

InterruptionExterne : configure les interruptions externes INT0, INT1 ou INT2. Le constructeur reçoit le numéro de l'interruption et le déclencheur voulu (`Declencheur` : niveau bas, changement, front descendant ou front montant), écrit les deux bits correspondants de EICRA, efface l'indicateur dans EIFR et active l'interruption dans EIMSK. `activer()`, `desactiver()`, `changerDeclencheur()` et `effacerDrapeau()` permettent de la modifier ensuite. La routine `ISR()` reste dans le code d'application.

Minuterie1 : délais avec la minuterie 1 en mode CTC et un diviseur d'horloge de 1024. `demarrer(dureeMs)` calcule la valeur de OCR1A en arithmétique entière, sans virgule flottante. On peut ensuite attendre par scrutation avec `estExpiree()`, bloquer avec `attendre(dureeMs)`, ou utiliser une interruption avec `activerInterruption()`.

Minuterie2 : génère une fréquence pour le piézo. La minuterie 2 est en mode CTC et inverse la broche OC2A (PD7) à chaque comparaison : le matériel produit un signal carré sans occuper le processeur. `genererFrequence(frequenceHz)` calcule OCR2A à partir de la formule f = F_CPU / (2 × 256 × (1 + OCR2A)), pour des fréquences de 62 Hz à 15 625 Hz. `arreter()` coupe le son.

Moteurs : génère le PWM des deux roues avec la minuterie 0 en mode PWM à phase correcte et un diviseur de 8, soit environ 1961 Hz. `ajusterRoueGauche()` et `ajusterRoueDroite()` reçoivent une vitesse de 0 à 255 et une direction (`Direction::AVANT` ou `Direction::ARRIERE`). `arreterRoueGauche()`, `arreterRoueDroite()` et `arreter()` mettent la vitesse à 0.

Uart : communication RS232 avec le PC à 2400 bauds, 8 bits, sans parité. On peut transmettre un octet, une chaîne de caractères ou un nombre converti en texte (`transmettreNombre()`), et recevoir un octet avec `recevoir()`. L'initialisation n'est faite qu'une fois, même si plusieurs objets Uart sont créés, pour ne pas modifier le débit pendant qu'un octet est en cours d'envoi.

debug.h (travail pratique 8) : la macro `DEBUG_PRINT(message, valeur)` envoie au PC un message suivi d'une valeur entière, grâce à la classe Uart. Lorsque `DEBUG` n'est pas défini, la macro devient `do {} while (0)`, du code mort que le compilateur élimine : aucun coût en mémoire ni en temps d'exécution.

```
#ifdef DEBUG
#define DEBUG_PRINT(message, valeur)                              \
    do {                                                          \
        Uart uart;                                                \
        uart.transmettre(message);                                \
        uart.transmettreNombre(static_cast<uint16_t>(valeur)); \
        uart.transmettre("\r\n");                                 \
    } while (0)
#else
#define DEBUG_PRINT(message, valeur) do {} while (0)
#endif
```

can et memoire_24 : le code de conversion analogique/numérique et de la mémoire externe

fourni par le cours est ajouté sans modification. Le Makefile compile tous les fichiers .cpp du répertoire, il n'a donc pas eu besoin d'être modifié pour les intégrer.

Programme de test

Le répertoire `tp/tp7/exec` contient `main.cpp`, qui utilise toutes les classes de la librairie. Il envoie d'abord un message au PC (Uart), affiche rouge, vert puis ambre (Del et Minuterie1), joue un La à 440 Hz puis à 880 Hz (Minuterie2), fait avancer, reculer puis tourner le robot (Moteurs), puis teste INT0 pendant 5 secondes : chaque appui sur le bouton change la couleur de la DEL (InterruptionExterne). Il lance ensuite la machine à états du problème 1 du travail pratique 2 : après trois appuis sur le bouton, la DEL s'allume en vert pendant deux secondes. La logique de transition et la logique de sortie sont dans deux switch séparés (machine de Moore). À chaque changement d'état, `DEBUG_PRINT` envoie l'état et le nombre d'appuis au PC, que l'on lit avec `serieViaUSB -l`. L'ensemble a été vérifié sur le robot, en mode normal et en mode débogage.

Partie 2 : Modifications apportées au Makefile de départ

Le Makefile de départ compile chaque fichier source en fichier objet (.o) avec `avr-gcc -c`, puis réunit les fichiers objets en un exécutable (.elf) par l'édition de liens, le convertit en .hex et le télécharge avec avrdude. Nous avons respecté le principe d'un seul Makefile par répertoire, chacun produisant une seule chose : la librairie dans `lib`, l'exécutable dans `exec`. Les règles de compilation `%.o: %.cpp` sont restées identiques dans les deux Makefiles.

2.1 Makefile de la librairie (lib/Makefile)

La compilation des fichiers .o ne change pas. Seule la dernière étape change : au lieu d'appeler l'éditeur de liens, on regroupe les fichiers objets dans une archive avec l'archiveur `avr-ar`.

- **Variables de l'archiveur.** Comme pour les autres commandes du Makefile, une variable donne la commande et une autre ses options : `AR=avr-ar` et `ARFLAGS=-crs`. L'option c crée l'archive si elle n'existe pas, r y insère ou remplace les fichiers objets, et s produit l'index des symboles qui permet à l'éditeur de liens de trouver rapidement les fonctions.
- **Cible.** `TRG=lib$(PROJECTNAME).a` avec `PROJECTNAME=robot` produit `librobot.a`. Le préfixe lib et l'extension .a sont la convention que l'option -l de l'éditeur de liens s'attend à trouver.
- **Commande de la règle $(TRG).** La dépendance `$(OBJDEPS)` ne change pas, seule la commande est remplacée :

```
# Avant (executable)
$(TRG): $(OBJDEPS)
	$(CC) $(LDFLAGS) -o $(TRG) $(OBJDEPS) -lm $(LIBS)

# Apres (librairie)
$(TRG): $(OBJDEPS)
	$(AR) $(ARFLAGS) $(TRG) $(OBJDEPS)
```

- **Sources.** `PRJSRC=$(wildcard *.cpp)` : tous les fichiers .cpp du répertoire font partie de la librairie. Ajouter un module, comme can.cpp ou memoire_24.cpp, ne demande aucune modification du Makefile, et chaque membre de l'équipe a pu ajouter ses fichiers sans conflit Git sur le Makefile.
- **Retraits.** Une librairie ne contient pas de main et ne se télécharge pas sur la carte. Nous avons retiré `LDFLAGS`, les variables du fichier .hex, la règle `%.hex: %.elf`, la règle

`install`, ainsi que `OBJCOPY` et `AVRDUDE`. La règle `all` dépend seulement de `$(TRG)`, et `clean` efface la librairie, les .o et les .d.

- **Fréquence d'horloge.** `-DF_CPU=8000000UL` est ajouté à `CFLAGS`, pour que la librairie et l'application utilisent la même valeur (nécessaire à `_delay_ms` et aux calculs des minuteries) sans la redéfinir dans chaque fichier.
- **Compilateurs récents.** Avec avr-gcc 12 et plus, le Makefile ajoute `--param=min-pagesize=0` à `CFLAGS`, ce qui évite de faux avertissements sur l'accès aux registres.

2.2 Makefile de l'exécutable (exec/Makefile)

Le Makefile de l'exécutable reste presque identique à celui de départ. Il faut seulement indiquer au compilateur où trouver les fichiers .h de la librairie, et à l'éditeur de liens où trouver la librairie elle-même :

```
REPERTOIRE_LIB=../lib
NOM_LIB=robot
INC=-I$(REPERTOIRE_LIB)
LIBS=-L$(REPERTOIRE_LIB) -l$(NOM_LIB)
```

- **INC (compilation).** `-I../lib` fait partie de `CFLAGS`, donc de la compilation de main.cpp en main.o. Le préprocesseur trouve ainsi `#include "del.h"` sans chemin relatif dans le code. Cette option ne sert plus à l'édition de liens, où il n'y a plus de fichiers sources.
- **LIBS (édition de liens).** `-L../lib` ajoute le répertoire de la librairie au chemin de recherche, et `-lrobot` demande de lier `librobot.a` ; l'éditeur de liens ajoute lui-même le préfixe lib et l'extension .a. La variable LIBS était déjà prévue dans la commande d'édition de liens du Makefile de départ, après les fichiers objets. Cet ordre compte : l'éditeur de liens cherche dans la librairie les symboles que main.o n'a pas encore résolus, et ne prend que les fichiers objets nécessaires.
- **Dépendance sur la librairie.** La règle `$(TRG)` dépend aussi de `$(REPERTOIRE_LIB)/lib$(NOM_LIB).a` : si la librairie est reconstruite, l'exécutable est lié de nouveau au prochain make. La librairie n'est jamais copiée dans exec, et elle n'est pas placée sous Git (fichier .gitignore), puisqu'on peut la reconstruire à tout moment.
- **Autres ajouts.** La même option `-DF_CPU=8000000UL` que dans la librairie, le même paramètre pour avr-gcc 12 et plus, et l'option `-P usb` dans la commande d'avrdude.

2.3 Cible debug (travail pratique 8)

Pour activer le mécanisme de débogage sans modifier de fichier, nous avons ajouté une cible `debug` au Makefile de l'exécutable :

```
.PHONY: all install clean debug

debug: CFLAGS += -DDEBUG
debug: clean all
```

La première ligne est une variable propre à la cible : avec `make debug`, `-DDEBUG` s'ajoute à `CFLAGS`, qui n'est utilisé que par les règles de compilation des .o. L'option est donc passée au compilateur, mais pas à l'éditeur de liens, où elle serait inutile. La deuxième ligne fait d'abord un `clean`, puis la construction habituelle : sans ce clean, make jugerait main.o à jour et ne le recompilerait pas avec `-DDEBUG`. La librairie n'est jamais compilée en mode débogage ; son Makefile n'a rien eu à changer pour le travail pratique 8.

Pour le vérifier, nous avons comparé les deux exécutables avec `avr-size` : 2 328 octets de code

en mode normal, contre 2 502 octets en mode débogage. Sur le robot, les messages d'état n'apparaissent dans serieViaUSB qu'en mode débogage, alors que le comportement reste identique : les appels à `DEBUG_PRINT` deviennent bien du code mort en mode normal.

2.4 Makefile racine et utilisation

Un Makefile dans `tp/tp7` ne compile rien lui-même : il appelle `make -C lib`, puis `make -C exec`, dans le bon ordre. Un simple `make` dans `tp/tp7` construit donc tout, et `make debug`, `make install` et `make clean` y sont aussi disponibles.

```
cd tp/tp7
make            # construit librobot.a, puis l'executable
make debug      # meme chose avec DEBUG_PRINT actif
make install    # telecharge le programme sur la carte
```

Le code compile sans erreur ni avertissement dans les deux modes.